Decidibilidade
Problemas que Nenhum Algoritmo Pode Resolver

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

Análise de Algoritmos – 2026.1

Roteiro da Aula

1. Revisão: P, NP e NP-Completo
2. Além da Complexidade
3. Programas e Decidibilidade
4. O Problema da Parada
5. Outros Problemas Indecidíveis
6. Reduções para Indecidibilidade
7. A Hierarquia Completa
8. Implicações e Resumo
9. Exercícios

O Que Já Estudamos

Classes de Complexidade – Resumo

Classe	Definição informal	Exemplos
P	Resolúvel em $O(n^k)$	BFS, Ordenação, IS-SORTED
NP	Verificável em $O(n^k)$	SAT, CLIQUE, MOCHILA 0-1
NP-Completo	Mais difícil de NP	SAT, 3-SAT, SUBSET-SUM

A Grande Questão em Aberto

$P \stackrel{?}{=} NP$ — Acreditamos que $P \neq NP$, mas ninguém provou.

Se $P = NP$: SAT seria polinomial e a criptografia moderna seria quebrada.

Ideia de NP-Completude

L é NP-completo se: (1) $L \in NP$ e (2) *todo* problema de NP se reduz a L.
Resolver L em tempo polinomial $\Rightarrow P = NP$.

Problemas NP-Completos Conhecidos

Alguns dos 21 Problemas de Karp (1972)

Problema	**Pergunta de decisão**
SAT / 3-SAT	Fórmula booleana (CNF) satisfatível?
CLIQUE	Existe subgrafo completo de tamanho k?
COLORAÇÃO ($k \geq 3$)	É possível colorir com $\leq k$ cores?
SUBSET-SUM	Existe subconjunto com soma $= k$?
PARTIÇÃO	Existe $S_1 \subseteq S$ com soma igual à metade da soma total de S?
MOCHILA 0-1	Seleção com peso $\leq C$ e valor $\geq V$?
TSP (decisão)	Existe tour com custo $\leq L$?

Ponte para a Aula de Hoje

Todos esses problemas são **difíceis**, mas têm algo em comum:
existe um algoritmo (lento) que os resolve.

Hoje: veremos problemas onde *nenhum* algoritmo os resolve corretamente.

Reduções Polinomiais – Recordando a Lógica

O que é uma Redução $A \leq_p B$?

Existe transformação f, em tempo polinomial, tal que:

$$x \in A \iff f(x) \in B.$$

Se B é fácil, então A também é. Se A é difícil, então B também é.

Exemplo Simples: CLIQUE $\leq_p$ CONJUNTO INDEPENDENTE

Dada uma instância (G, k) de CLIQUE, construa $f(G, k) = (\overline{G}, k)$, onde $\overline{G}$ é o grafo complemento de G.
Em $\overline{G}$, dois vértices são adjacentes sse não eram em G.
Logo, G tem uma clique de tamanho $k \iff \overline{G}$ tem um conjunto independente de tamanho k.
Custo: construir $\overline{G}$ em tempo polinomial (por exemplo, $O(|V|^2)$). ■

Reduções Polinomiais – Erro Clássico

Direção da Redução – Erro Clássico

$A_{\text{difícil}} \leq_p B \quad \Rightarrow B$ é difícil

$B \leq_p A_{\text{difícil}} \quad \Rightarrow$ nada se conclui sobre B

Uma Pergunta Mais Fundamental

Até Agora Perguntávamos...

"Quão **rápido** um algoritmo pode resolver este problema?"
Respostas: $O(n)$, $O(n^2)$, $O(2^n)$, ...

Hoje Perguntamos...

"**Existe algum algoritmo** que resolva este problema — mesmo que lento?"

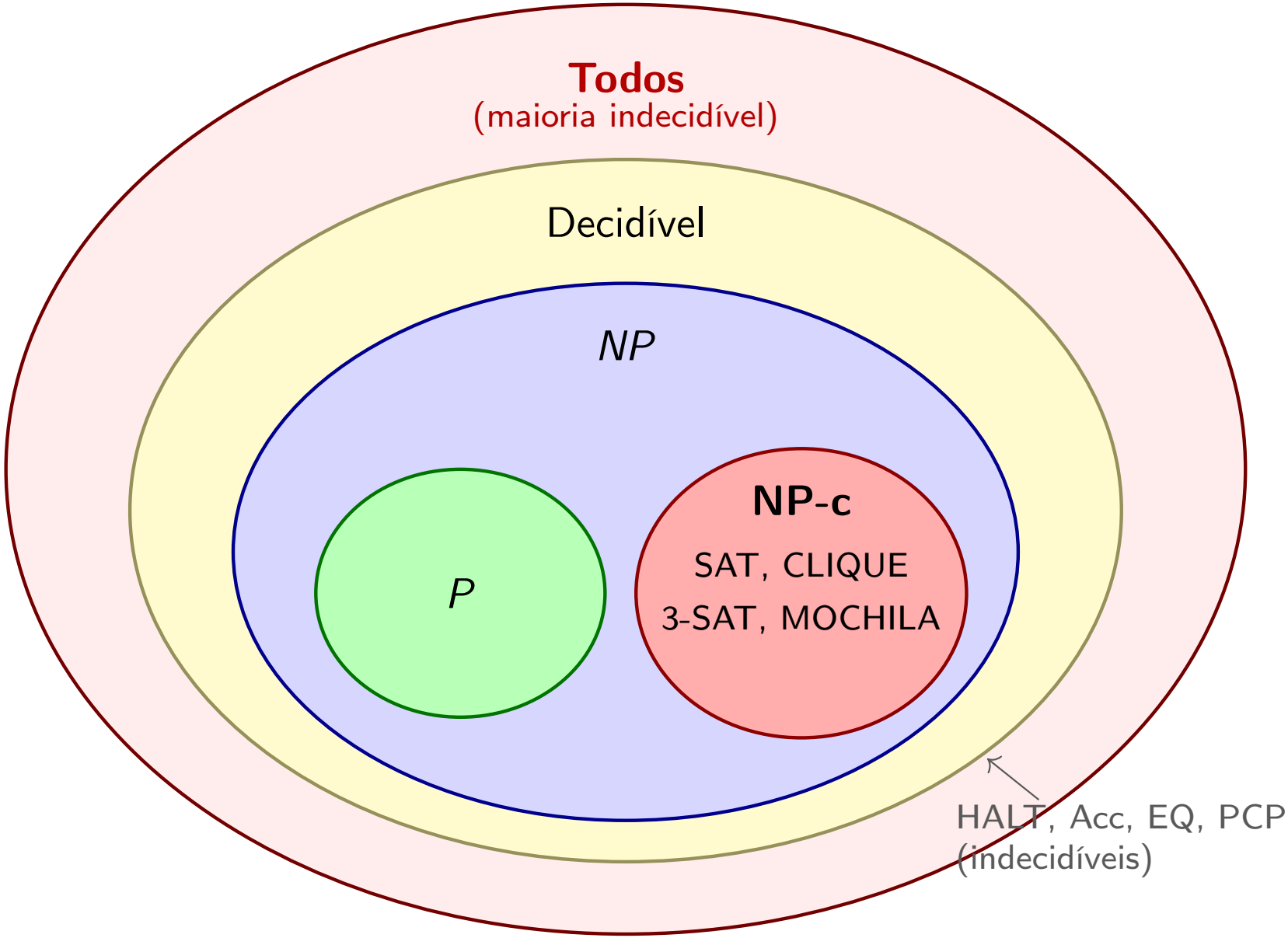
Pode parecer óbvio que sempre existe... mas **não é verdade**.

Problemas sem Solução Algorítmica Possível

- **Problema da Parada:** dado o código de um programa P e uma entrada x, a execução de $P(x)$ termina alguma vez?
- **Equivalência de Programas:** dois programas P_1 e P_2 produzem sempre a mesma saída?
- **Parada Universal:** o programa P para para *toda* entrada possível?

Nenhum desses tem solução algorítmica — nem com tempo ilimitado!

A Hierarquia Completa dos Problemas



Programas: Três Comportamentos Possíveis

O Que um Algoritmo Pode Fazer com uma Entrada

Quando executamos um programa P com entrada x, exatamente uma de três coisas acontece:

- $P(x)$ **para e aceita** (retorna SIM / resposta positiva).
- $P(x)$ **para e rejeita** (retorna NÃO / resposta negativa).
- $P(x)$ **nunca para** — loop infinito, sem resposta alguma.

Código de um Programa como Dado

O código-fonte de qualquer programa é uma string de caracteres.

Portanto: podemos passar **o código de um programa como entrada para outro programa**.

Exemplo: um compilador recebe código-fonte como entrada.
Exemplo: um interpretador Python recebe um script `.py` como entrada.

A Pergunta Central da Decidibilidade

Dado o código de um programa P e uma entrada x: **a execução de $P(x)$ termina alguma vez?**

Essa é exatamente a pergunta do **Problema da Parada**.

Decidível vs Indecidível

Definições

Decidível: existe um algoritmo que, para *toda* entrada, para e responde corretamente SIM ou NÃO.

Indecidível: *nenhum* algoritmo responde corretamente em todos os casos.

Tipo	Responde SIM	Responde NÃO	Exemplo
Decidível	**sempre**	**sempre**	IS-SORTED, SAT
Indecidível	pode errar/loopar	pode errar/loopar	HALT, Acc

Atenção ao Loop

Um programa que entra em loop infinito **não dá resposta nenhuma**.
Um decididor *nunca* pode entrar em loop: sempre para com SIM ou NÃO.

Analogia

Decididor = professor que sempre responde "certo" ou "errado" para qualquer questão.
Indecidível = não existe professor algum que sempre acerte.

O Problema da Parada – Enunciado

HALT (Problema da Parada)

Entrada: o código de um programa P e uma entrada x.

Pergunta: a execução de $P(x)$ **termina** em algum momento?

Casos possíveis

Entrada	**Comportamento de $P(x)$**	**Resposta**
`fatorial`, $x = 5$	Para, retorna 120	SIM
`while True: pass`, $x = 0$	Loop infinito	NÃO
`collatz`, $x = 27$	Para após 111 passos	SIM
`collatz`, $x =?$	Não se sabe (conjectura)	???

O Que Seria um Solucionador de HALT?

Um programa H que recebe $\langle P, x \rangle$ e:

- Retorna **SIM** se $P(x)$ para.
- Retorna **NÃO** se $P(x)$ não para.
- Sempre responde em tempo finito.

Teorema (Turing, 1936): tal programa H não pode existir.

Conjectura de Collatz – O Problema

Regra de Collatz (problema 3n+1)

Comece com um inteiro $n > 0$ e repita:

- Se n é par, faça $n \leftarrow n/2$.
- Se n é ímpar, faça $n \leftarrow 3n + 1$.

Pergunta: esse processo sempre chega a 1?

Exemplos

Sequência para $n = 6$: $6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$.
Para $n = 6$, para em tempo finito. Já para $n = 27$, só chega em 1 após 111 passos.

Conjectura de Collatz – Por que é Difícil?

Por que aparece nesta aula?

Collatz ilustra a diferença entre:

- testar muitos casos (computacionalmente possível),
- provar para todos os casos (matematicamente difícil).

O estado atual

- Não existe, até hoje, um n conhecido que **nunca** chegue a 1.
- Exemplo de trajetória longa: $n = 27$ leva 111 passos até 1.
- Exemplo famoso: $n = 837799$ leva 524 passos e sobe até 2 974 984 antes de descer.
- Mesmo assim, falta uma prova para *todo* $n > 0$.

Conexão com HALT

A linha "`collatz`, $x =?$" indica: falta uma prova geral para todos os casos.

Por que HALT Parece Difícil? – Exemplos (1/2)

Exemplo 1: Para sempre (fácil de analisar)

```
1  funcao soma(n):
2    total ← 0
3    para i de 1 ate n faca
4      total ← total + i
5    fim
6    retorne total
```

Para qualquer $n \geq 0$: o laço executa exatamente n vezes. HALT retornaria SIM.

Exemplo 2: Loop infinito óbvio

```
1  funcao loopa(x):
2    enquanto verdadeiro faca
3      x ← x + 1
4    fim
```

Nunca para. HALT retornaria NÃO.

Observação: nesses dois casos, a própria estrutura do código já sugere a resposta. No próximo exemplo, isso deixa de ser evidente.

Por que HALT Parece Difícil? – Exemplos (2/2)

Exemplo 3: Conjectura de Collatz – Difícil de analisar

```
1  funcao collatz(n):
2    enquanto n ≠ 1 faca
3      se n mod 2 = 0:   n ← n / 2
4      senao:            n ← 3*n + 1
5    fim
6    retorne 1
```

- Para $n = 27$: a sequência tem **111 passos** antes de chegar a 1.
- Para $n = 871$: atinge o pico de 190 996 antes de descer para 1.
- Nunca foi encontrado nenhum $n > 0$ que entre em loop.

Conjectura de Collatz (em aberto desde 1937)

collatz(n) para para todo $n > 0$?

Acredita-se que sim, mas **nenhuma prova existe até hoje**.
Um solucionador de HALT teria que responder isso instantaneamente — para qualquer n.

Isso ilustra por que HALT é impossível em geral.

Uma Historinha: O Paradoxo do Barbeiro

As Regras da Cidade

- Na cidade existe **um único barbeiro** (um homem).
- **Todo homem** deve se manter barbeado: ou vai ao barbeiro, ou faz a própria barba.
- O barbeiro faz a barba **exatamente** de quem **não** faz a própria barba.

Uma Historinha: O Paradoxo do Barbeiro

As Regras da Cidade

- Na cidade existe **um único barbeiro** (um homem).
- **Todo homem** deve se manter barbeado: ou vai ao barbeiro, ou faz a própria barba.
- O barbeiro faz a barba **exatamente** de quem **não** faz a própria barba.

Pergunta: quem faz a barba do barbeiro?

O barbeiro também é um homem, então ele também precisa estar barbeado.

Uma Historinha: O Paradoxo do Barbeiro

As Regras da Cidade

- Na cidade existe **um único barbeiro** (um homem).
- **Todo homem** deve se manter barbeado: ou vai ao barbeiro, ou faz a própria barba.
- O barbeiro faz a barba **exatamente** de quem **não** faz a própria barba.

Pergunta: quem faz a barba do barbeiro?

O barbeiro também é um homem, então ele também precisa estar barbeado.

Caso 1: ele faz a própria barba

Mas o barbeiro só barbeia quem **não** faz a própria barba.

Então o barbeiro **não pode** barbear a si mesmo. Contradição!

Uma Historinha: O Paradoxo do Barbeiro

As Regras da Cidade

- Na cidade existe **um único barbeiro** (um homem).
- **Todo homem** deve se manter barbeado: ou vai ao barbeiro, ou faz a própria barba.
- O barbeiro faz a barba **exatamente** de quem **não** faz a própria barba.

Pergunta: quem faz a barba do barbeiro?

O barbeiro também é um homem, então ele também precisa estar barbeado.

Caso 1: ele faz a própria barba

Mas o barbeiro só barbeia quem **não** faz a própria barba.

Então o barbeiro **não pode** barbear a si mesmo. Contradição!

Caso 2: ele não faz a própria barba

Então ele vai ao barbeiro... que é **ele mesmo**.

Logo ele faz a própria barba. Contradição!

Uma Historinha: O Paradoxo do Barbeiro

As Regras da Cidade

- Na cidade existe **um único barbeiro** (um homem).
- **Todo homem** deve se manter barbeado: ou vai ao barbeiro, ou faz a própria barba.
- O barbeiro faz a barba **exatamente** de quem **não** faz a própria barba.

Pergunta: quem faz a barba do barbeiro?

O barbeiro também é um homem, então ele também precisa estar barbeado.

Caso 1: ele faz a própria barba

Mas o barbeiro só barbeia quem **não** faz a própria barba.

Então o barbeiro **não pode** barbear a si mesmo. Contradição!

Caso 2: ele não faz a própria barba

Então ele vai ao barbeiro... que é **ele mesmo**.

Logo ele faz a própria barba. Contradição!

Não há saída: um barbeiro com essas regras **não pode existir**.

Outro Paradoxo – e o Que Eles Têm em Comum

Duas Frases

(1) "A frase seguinte é verdadeira." **(2)** "A frase anterior é falsa."

Outro Paradoxo – e o Que Eles Têm em Comum

Duas Frases

(1) "A frase seguinte é verdadeira." **(2)** "A frase anterior é falsa."

- Se (1) é verdadeira $\Rightarrow$ (2) é verdadeira $\Rightarrow$ (1) é falsa. Contradição!
- Se (1) é falsa $\Rightarrow$ (2) é falsa $\Rightarrow$ (1) é verdadeira. Contradição!

Outro Paradoxo – e o Que Eles Têm em Comum

Duas Frases

(1) "A frase seguinte é verdadeira." **(2)** "A frase anterior é falsa."

- Se (1) é verdadeira $\Rightarrow$ (2) é verdadeira $\Rightarrow$ (1) é falsa. Contradição!
- Se (1) é falsa $\Rightarrow$ (2) é falsa $\Rightarrow$ (1) é verdadeira. Contradição!

O Ingrediente dos Dois Paradoxos

1. **Auto-referência:** algo que fala sobre si mesmo
   (o barbeiro barbeando a si mesmo; a frase falando de si mesma).
2. **Inversão:** uma regra que manda fazer **o contrário**
   (barbeia quem *não* se barbeia; a frase diz que a outra é *falsa*).

Outro Paradoxo – e o Que Eles Têm em Comum

Duas Frases

(1) "A frase seguinte é verdadeira." **(2)** "A frase anterior é falsa."

- Se (1) é verdadeira ⇒ (2) é verdadeira ⇒ (1) é falsa. Contradição!
- Se (1) é falsa ⇒ (2) é falsa ⇒ (1) é verdadeira. Contradição!

O Ingrediente dos Dois Paradoxos

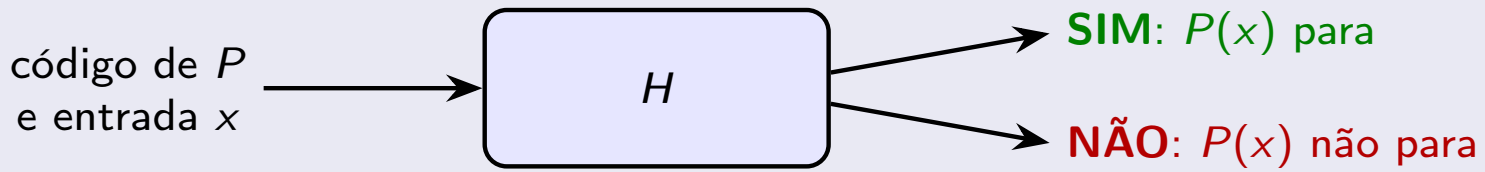
1. **Auto-referência:** algo que fala sobre si mesmo
   (o barbeiro barbeando a si mesmo; a frase falando de si mesma).
2. **Inversão:** uma regra que manda fazer **o contrário**
   (barbeia quem *não* se barbeia; a frase diz que a outra é *falsa*).

Guarde essa ideia

É exatamente essa receita (**auto-referência** + **fazer o contrário**) que vamos usar para provar que o Problema da Parada não tem solução.

Voltando ao Problema da Parada: O Que Queremos?

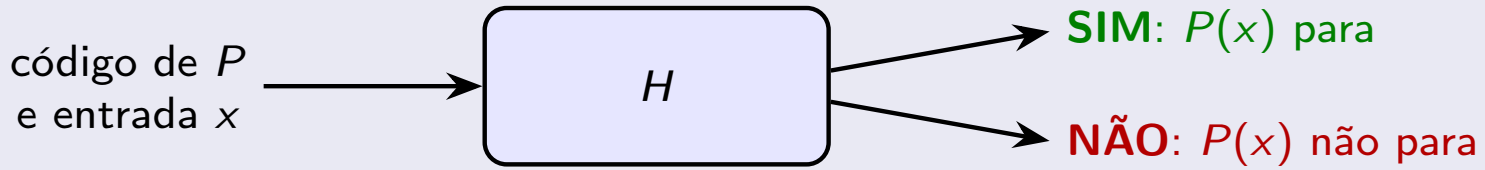
Queremos um programa H que resolva HALT



Exigência: H **sempre** termina e responde — para *qualquer* par (P, x).

Voltando ao Problema da Parada: O Que Queremos?

Queremos um programa H que resolva HALT



Exigência: H **sempre** termina e responde — para *qualquer* par (P, x).

Lembrete: o que pode acontecer quando executamos $P(x)$

- $P(x)$ **para** e responde SIM;
- $P(x)$ **para** e responde NÃO;
- $P(x)$ **nunca para** (loop infinito).

O terceiro caso é o traiçoeiro: a execução **nunca avisa** que entrou em loop. Ela simplesmente continua rodando.

Primeira Tentativa: Simular o Programa

Ideia ingênua: executar $P(x)$ e ver o que acontece

Não é estranho: um **interpretador Python** recebe um script e o executa.

```
1  funcao U(P, x):
2    simule P(x)      // executa P sobre x, passo a passo
3    retorne SIM      // so chega aqui se P(x) parou
```

Primeira Tentativa: Simular o Programa

Ideia ingênua: executar $P(x)$ e ver o que acontece

Não é estranho: um **interpretador Python** recebe um script e o executa.

```
1  funcao U(P, x):
2    simule P(x)      // executa P sobre x, passo a passo
3    retorne SIM      // so chega aqui se P(x) parou
```

Se $P(x)$...	**então $U(P,x)$...**	
para	responde SIM	correto
nunca para	**também nunca para**	sem resposta!

Primeira Tentativa: Simular o Programa

Ideia ingênua: executar $P(x)$ e ver o que acontece

Não é estranho: um **interpretador Python** recebe um script e o executa.

```
1  funcao U(P, x):
2    simule P(x)      // executa P sobre x, passo a passo
3    retorne SIM      // so chega aqui se P(x) parou
```

Se $P(x)$...	**então $U(P,x)$...**	
para	responde SIM	correto
nunca para	**também nunca para**	sem resposta!

O problema de U

U acerta todos os casos em que a resposta é SIM, mas **fica preso** nos casos em que a resposta é NÃO. Logo U **não resolve** HALT.

"E se esperarmos bastante?" Após 1 hora rodando, nada garante que $P(x)$ não pararia no minuto seguinte.

Por isso o nome **Problema da Parada**: toda a dificuldade está em saber **se o programa vai parar**.

Estratégia: Prova por Contradição

Suponha que H existe

Vamos **supor** que alguém conseguiu escrever o programa H ("H" de **hipotético**):

- $H(P, x) =$ **SIM** se $P(x)$ para;
- $H(P, x) =$ **NÃO** se $P(x)$ não para;
- H **sempre** responde em tempo finito (nunca entra em loop).

Estratégia: Prova por Contradição

Suponha que H existe

Vamos **supor** que alguém conseguiu escrever o programa H ("H" de **hipotético**):

- $H(P, x) =$ **SIM** se $P(x)$ para;
- $H(P, x) =$ **NÃO** se $P(x)$ não para;
- H **sempre** responde em tempo finito (nunca entra em loop).

O plano

1. Usar H como **peça** para montar um novo programa, que chamaremos de D.
2. Mostrar que D leva a um **paradoxo** (como o do barbeiro).
3. Como D é fácil de montar *a partir de* H, a culpa do paradoxo é de H: H **não pode existir.**

Antes: Programas que Recebem Programas

Código é texto

O código-fonte de um programa é só uma sequência de caracteres.
Então ele pode ser **entrada** de outro programa — e até **do próprio programa**.

Antes: Programas que Recebem Programas

Código é texto

O código-fonte de um programa é só uma sequência de caracteres.
Então ele pode ser **entrada** de outro programa — e até **do próprio programa**.

Isso acontece na prática

- Compiladores de C são escritos em C. Um compilador recebe programas C... e ele mesmo **é** um programa C. Então podemos **compilar o compilador com ele mesmo**.
- Existem interpretadores de Python escritos em Python.
- Um programa que conta quantas linhas tem o código recebido pode receber o **próprio código**.

Antes: Programas que Recebem Programas

Código é texto

O código-fonte de um programa é só uma sequência de caracteres.
Então ele pode ser **entrada** de outro programa — e até **do próprio programa**.

Isso acontece na prática

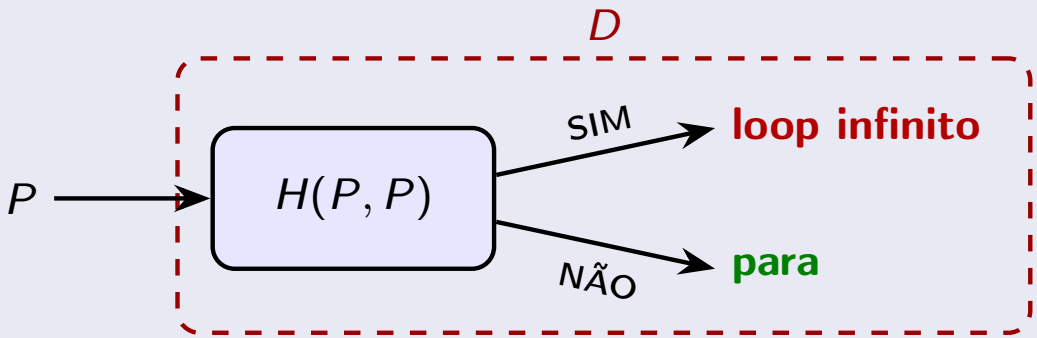
- Compiladores de C são escritos em C. Um compilador recebe programas C... e ele mesmo **é** um programa C. Então podemos **compilar o compilador com ele mesmo**.
- Existem interpretadores de Python escritos em Python.
- Um programa que conta quantas linhas tem o código recebido pode receber o **próprio código**.

Notação

$P(P)$ = executar o programa P dando como entrada **o próprio código de** P.
Logo $H(P, P)$ pergunta: **"P, rodando sobre o próprio código, para?"**

Construindo o Programa D

D recebe um único programa P e faz o contrário do que H prevê




```
1  funcao D(P):                    // recebe so um programa
2    se H(P, P) = SIM              // H diz que P(P) para?
3      enquanto verdadeiro faca    // entao D entra em loop
4      fim
5    senao                         // H diz que P(P) nao para?
6      retorne SIM                 // entao D para
7    fim
```

H diz que $P(P)$ **para** $\Rightarrow$ $D(P)$ **não para**.
H diz que $P(P)$ **não para** $\Rightarrow$ $D(P)$ **para**.

Resumindo Até Agora

H (hipotético)

- Recebe **um programa e uma entrada**: (P, x).
- Responde **SIM** quando $P(x)$ para.
- Responde **NÃO** quando $P(x)$ não para.
- Sempre termina.

D (construído a partir de H)

- Recebe **um único programa**: P.
- **Para** quando $P(P)$ **não** para.
- **Entra em loop** quando $P(P)$ para.

A essência de D

D faz **exatamente o contrário** do que P faz quando recebe o próprio código.

Compare com o barbeiro: ele barbeia **exatamente** quem **não** se barbeia.

Uma Tabela de Programas

Coloque nas linhas os programas e nas colunas os **códigos** desses mesmos programas. A célula (P_i, P_j) mostra o que acontece ao executar $P_i(P_j)$:

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	**para**		**para**		$\cdots$
P_2	**para**	**para**	**para**	**para**	$\cdots$
P_3					$\cdots$
P_4	**para**	**para**			$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

E as células em branco?

São execuções que **ainda estão rodando**. Não sabemos se vão parar daqui a pouco ou se estão em loop infinito — olhando só a execução, **não temos como saber**.

Exemplo: P_3 não para (até onde vimos) com nenhum código; P_2 para com todos.

A Mesma Tabela, Agora Perguntando para H

Como H **sempre** responde, ela preenche **todas** as células: $H(P_i, P_j)$.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	**SIM**				$\cdots$
P_2					$\cdots$
P_3					$\cdots$
P_4					$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Preenchendo

- $H(P_1, P_1)$: na tabela anterior, $P_1(P_1)$ **para** $\Rightarrow$ H responde **SIM**.

A Mesma Tabela, Agora Perguntando para H

Como H **sempre** responde, ela preenche **todas** as células: $H(P_i, P_j)$.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	**SIM**	**NÃO**			$\cdots$
P_2					$\cdots$
P_3					$\cdots$
P_4					$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Preenchendo

- $H(P_1, P_1)$: na tabela anterior, $P_1(P_1)$ **para** $\Rightarrow$ H responde **SIM**.
- $H(P_1, P_2)$: $P_1(P_2)$ estava em branco — mas H **sabe** que não para $\Rightarrow$ **NÃO**.

A Mesma Tabela, Agora Perguntando para H

Como H **sempre** responde, ela preenche **todas** as células: $H(P_i, P_j)$.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	**SIM**	**NÃO**	**SIM**	**NÃO**	$\cdots$
P_2	**SIM**	**SIM**	**SIM**	**SIM**	$\cdots$
P_3	**NÃO**	**NÃO**	**NÃO**	**NÃO**	$\cdots$
P_4	**SIM**	**SIM**	**NÃO**	**NÃO**	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Preenchendo

- $H(P_1, P_1)$: na tabela anterior, $P_1(P_1)$ **para** $\Rightarrow$ H responde **SIM**.
- $H(P_1, P_2)$: $P_1(P_2)$ estava em branco — mas H **sabe** que não para $\Rightarrow$ **NÃO**.
- E assim por diante: **nenhuma célula fica em branco**.

O Que D Faz com Cada Programa?

$D(P_i)$ consulta $H(P_i, P_i)$ — ou seja, só olha a **diagonal** da tabela — e faz o contrário.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	SIM	NÃO	SIM	NÃO	$\cdots$
P_2	SIM	SIM	SIM	SIM	$\cdots$
P_3	NÃO	NÃO	NÃO	NÃO	$\cdots$
P_4	SIM	SIM	NÃO	NÃO	$\cdots$

Seguindo a definição de D

O Que D Faz com Cada Programa?

$D(P_i)$ consulta $H(P_i, P_i)$ — ou seja, só olha a **diagonal** da tabela — e faz o contrário.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	SIM	NÃO	SIM	NÃO	$\cdots$
P_2	SIM	SIM	SIM	SIM	$\cdots$
P_3	NÃO	NÃO	NÃO	NÃO	$\cdots$
P_4	SIM	SIM	NÃO	NÃO	$\cdots$

Seguindo a definição de D

- $D(P_1)$: $H(P_1, P_1) = \text{SIM}$ ($P_1(P_1)$ para) $\Rightarrow$ $D(P_1)$ **loop**.

O Que D Faz com Cada Programa?

$D(P_i)$ consulta $H(P_i, P_i)$ — ou seja, só olha a **diagonal** da tabela — e faz o contrário.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	SIM	NÃO	SIM	NÃO	$\cdots$
P_2	SIM	SIM	SIM	SIM	$\cdots$
P_3	NÃO	NÃO	NÃO	NÃO	$\cdots$
P_4	SIM	SIM	NÃO	NÃO	$\cdots$

Seguindo a definição de D

- $D(P_1)$: $H(P_1, P_1) = \text{SIM}$ ($P_1(P_1)$ para) $\Rightarrow$ $D(P_1)$ **loop**.
- $D(P_2)$: $H(P_2, P_2) = \text{SIM}$ ($P_2(P_2)$ para) $\Rightarrow$ $D(P_2)$ **loop**.

O Que D Faz com Cada Programa?

$D(P_i)$ consulta $H(P_i, P_i)$ — ou seja, só olha a **diagonal** da tabela — e faz o contrário.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	SIM	NÃO	SIM	NÃO	$\cdots$
P_2	SIM	SIM	SIM	SIM	$\cdots$
P_3	NÃO	NÃO	NÃO	NÃO	$\cdots$
P_4	SIM	SIM	NÃO	NÃO	$\cdots$

Seguindo a definição de D

- $D(P_1)$: $H(P_1, P_1) =$ SIM ($P_1(P_1)$ para) $\Rightarrow$ $D(P_1)$ **loop**.
- $D(P_2)$: $H(P_2, P_2) =$ SIM ($P_2(P_2)$ para) $\Rightarrow$ $D(P_2)$ **loop**.
- $D(P_3)$: $H(P_3, P_3) =$ NÃO ($P_3(P_3)$ não para) $\Rightarrow$ $D(P_3)$ **para**.

O Que D Faz com Cada Programa?

$D(P_i)$ consulta $H(P_i, P_i)$ — ou seja, só olha a **diagonal** da tabela — e faz o contrário.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$
P_1	SIM	NÃO	SIM	NÃO	$\cdots$
P_2	SIM	SIM	SIM	SIM	$\cdots$
P_3	NÃO	NÃO	NÃO	NÃO	$\cdots$
P_4	SIM	SIM	NÃO	NÃO	$\cdots$

Seguindo a definição de D

- $D(P_1)$: $H(P_1, P_1) =$ SIM ($P_1(P_1)$ para) $\Rightarrow$ $D(P_1)$ **loop**.
- $D(P_2)$: $H(P_2, P_2) =$ SIM ($P_2(P_2)$ para) $\Rightarrow$ $D(P_2)$ **loop**.
- $D(P_3)$: $H(P_3, P_3) =$ NÃO ($P_3(P_3)$ não para) $\Rightarrow$ $D(P_3)$ **para**.
- $D(P_4)$: $H(P_4, P_4) =$ NÃO ($P_4(P_4)$ não para) $\Rightarrow$ $D(P_4)$ **para**.

A Parte Legal: D Recebendo o Próprio Código

D também é um programa

Então podemos executar D passando como entrada **o código do próprio** D: $D(D)$. Pela definição, $D(D)$ consulta $H(D, D)$ e faz o contrário.

A Parte Legal: D Recebendo o Próprio Código

D também é um programa

Então podemos executar D passando como entrada **o código do próprio** D: $D(D)$. Pela definição, $D(D)$ consulta $H(D, D)$ e faz o contrário.

Se $H(D, D) = \text{SIM}$

H afirma: "$D(D)$ **para**".

Mas, pela definição, D então entra em **loop**.

$D(D)$ **não para**. Contradição!

A Parte Legal: D Recebendo o Próprio Código

D também é um programa

Então podemos executar D passando como entrada **o código do próprio** D: $D(D)$. Pela definição, $D(D)$ consulta $H(D, D)$ e faz o contrário.

Se $H(D, D) =$ SIM

H afirma: "$D(D)$ **para**".

Mas, pela definição, D então entra em **loop**.

$D(D)$ **não para**. Contradição!

Se $H(D, D) =$ NÃO

H afirma: "$D(D)$ **não para**".

Mas, pela definição, D então **para**.

$D(D)$ **para**. Contradição!

A Parte Legal: D Recebendo o Próprio Código

D também é um programa

Então podemos executar D passando como entrada **o código do próprio** D: $D(D)$.
Pela definição, $D(D)$ consulta $H(D, D)$ e faz o contrário.

Se $H(D, D) =$ SIM

H afirma: "$D(D)$ **para**".

Mas, pela definição, D então entra em **loop**.

$D(D)$ **não para**. Contradição!

Se $H(D, D) =$ NÃO

H afirma: "$D(D)$ **não para**".

Mas, pela definição, D então **para**.

$D(D)$ **para**. Contradição!

Em uma frase

$$D(D) \textbf{ para} \iff D(D) \textbf{ não para}.$$

É o barbeiro de novo: "o barbeiro se barbeia $\iff$ o barbeiro não se barbeia".

Vendo a Contradição na Tabela

A tabela lista **todos** os programas. Como D é um programa, ele aparece como **linha** e como **coluna**.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$	**cód.** D
P_1	**para**	**loop**	**para**	**loop**	$\cdots$	$\cdots$
P_2	**para**	**para**	**para**	**para**	$\cdots$	$\cdots$
P_3	**loop**	**loop**	**loop**	**loop**	$\cdots$	$\cdots$
P_4	**para**	**para**	**loop**	**loop**	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
D						

Lendo a tabela

Vendo a Contradição na Tabela

A tabela lista **todos** os programas. Como D é um programa, ele aparece como **linha** e como **coluna**.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$	**cód.** D
P_1	para	loop	para	loop	$\cdots$	$\cdots$
P_2	para	para	para	para	$\cdots$	$\cdots$
P_3	loop	loop	loop	loop	$\cdots$	$\cdots$
P_4	para	para	loop	loop	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
D	loop	loop	para	para	$\cdots$	

Lendo a tabela

- A linha de D é o **oposto da diagonal**: onde a diagonal diz "para", D entra em loop, e vice-versa.

Vendo a Contradição na Tabela

A tabela lista **todos** os programas. Como D é um programa, ele aparece como **linha** e como **coluna**.

	cód. P_1	**cód.** P_2	**cód.** P_3	**cód.** P_4	$\cdots$	**cód.** D
P_1	para	loop	para	loop	$\cdots$	$\cdots$
P_2	para	para	para	para	$\cdots$	$\cdots$
P_3	loop	loop	loop	loop	$\cdots$	$\cdots$
P_4	para	para	loop	loop	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
D	loop	loop	para	para	$\cdots$	???

Lendo a tabela

- A linha de D é o **oposto da diagonal**: onde a diagonal diz "para", D entra em loop, e vice-versa.
- Na célula (D, D), D precisa fazer o oposto... **dela mesma**. Não existe valor possível para essa célula!

Conclusão: HALT é Indecidível

Recapitulando a prova

1. **Supomos** que existe H, que sempre responde se $P(x)$ para.
2. Usando H, **construímos** D: recebe P e faz o contrário do que $P(P)$ faz.
3. Executamos $D(D)$ e chegamos a: $D(D)$ para $\iff$ $D(D)$ não para. Absurdo!
4. D é só "H + um se + um loop". Se D não pode existir, H **não pode existir**.

Teorema (Turing, 1936)

HALT é indecidível: não existe nenhum algoritmo que, para todo programa P e entrada x, sempre termine e responda corretamente se $P(x)$ para. ■

O que isso **não** significa

- Não é falta de tempo ou de computador rápido: é uma **impossibilidade lógica**.
- Para *muitos* programas específicos dá para responder (ex.: `soma`, `loopa`). O impossível é um único algoritmo que acerte **todos** os casos.

Acc – O Problema da Aceitação

Acc – Acceptance Problem

$$\text{Acc} = \{\langle P, x\rangle \mid P \text{ é um programa e } P \text{ aceita } x\}$$

Pergunta: dado o código de um programa P e uma string x, P aceita x?

Relação com HALT

Acc e HALT são problemas próximos e ambos indecidíveis:

- HALT pergunta: $P(x)$ *para*?
- Acc pergunta: $P(x)$ *aceita* (para e responde SIM)?

Se $P(x)$ aceita, certamente $P(x)$ parou. Portanto HALT $\leq_m$ Acc e Acc $\leq_m$ HALT — ambos equivalentes em dificuldade.

Acc é Indecidível – Redução a partir de HALT

Se existisse decididor V para Acc, resolveríamos HALT:

Dado (P, x) (instância de HALT), construa $P'(y)$:

- Executa $P(x)$ (ignora y); se $P(x)$ parar, aceita y.

$P(x)$ para $\iff$ P' aceita qualquer y $\iff$ V aceita $\langle P', y\rangle$.

Se V existisse, resolveríamos HALT. Contradição. ■

Problema de Correspondência de Post (PCP)

Enunciado

Dado: k pares de strings $(u_1, v_1), \ldots, (u_k, v_k)$.

Pergunta: existe sequência de índices $i_1, i_2, \ldots, i_m$ (com repetição permitida) tal que

$$u_{i_1}\, u_{i_2} \cdots u_{i_m} = v_{i_1}\, v_{i_2} \cdots v_{i_m}?$$

Instância Concreta com solução

Par	u_i	v_i
1	ab	a
2	b	ab
3	a	b

Sequência $(1, 3, 2)$: u-concatenado $=$ ab$|$a$|$b $=$ abab. v-concatenado $=$ a$|$b$|$ab $=$ abab.
Iguais! ✓

Resultado

Apesar de parecer um puzzle simples, PCP é **indecidível** em geral.
Prova: redução de HALT para PCP (via construção técnica).

Importância: PCP é usado como ponto de partida para provar indecidibilidade de outros problemas em linguagens formais, gramáticas e lógica.

Estratégia: Reduções a partir de HALT

Análogo à NP-Completude, Porém Mais Grave

Para provar que um novo problema X é **indecidível**:

1. Tome um problema **já sabidamente indecidível** (ex.: HALT ou Acc).
2. Construa uma redução f que transforma qualquer instância de HALT em instância de X.
3. Mostre que f preserva SIM/NÃO.

Conclusão: se X fosse decidível, poderíamos decidir HALT — contradição.

Exemplo: "P para para toda entrada" é indecidível

Problema X: dado código de P, P para para **toda** entrada?

Redução de HALT: dado (P_0, x_0), construa P':

```
1  funcao P'(qualquerCoisa):  // ignora a entrada
2    P0(x0)                   // executa P0 sobre x0 (entrada fixa)
3    retorne SIM
```

Chave: P' para para *toda* entrada $\iff$ $P_0(x_0)$ para.

Portanto: decididor para $X \Rightarrow$ decididor para HALT. Contradição. ■

Exemplo Detalhado: Equivalência de Programas

Problema EQ (Equivalência de Programas)

Pergunta: os programas P_1 e P_2 se comportam **identicamente** para toda entrada?

Prova de Indecidibilidade (por redução de HALT)

Dado (P, x) (instância de HALT), construa:

- P_1: rejeita tudo (nunca aceita nada).
- $P_2(y)$: executa $P(x)$ (ignora y); se $P(x)$ parar, aceita y.

Se $P(x)$ para: P_2 aceita tudo $\neq P_1$ que rejeita tudo. Portanto $P_1 \not\equiv P_2$.
Se $P(x)$ não para: P_2 nunca aceita nada, igual a P_1. Portanto $P_1 \equiv P_2$.

$P(x)$ para $\iff$ $P_1 \not\equiv P_2$.

Decididor para EQ $\Rightarrow$ decididor para HALT. Contradição. ■

Consequência Prática

Não existe ferramenta automática que determine se dois programas são equivalentes.
Refatoração automática com garantia de correção semântica é impossível em geral.

A Hierarquia de Decidibilidade

Classe	Responde SIM	Responde NÃO	Exemplos
Decidível	✓	✓	SAT, CLIQUE, IS-SORTED
Indecidível	pode errar/loop	pode errar/loop	HALT, Acc, EQ, PCP

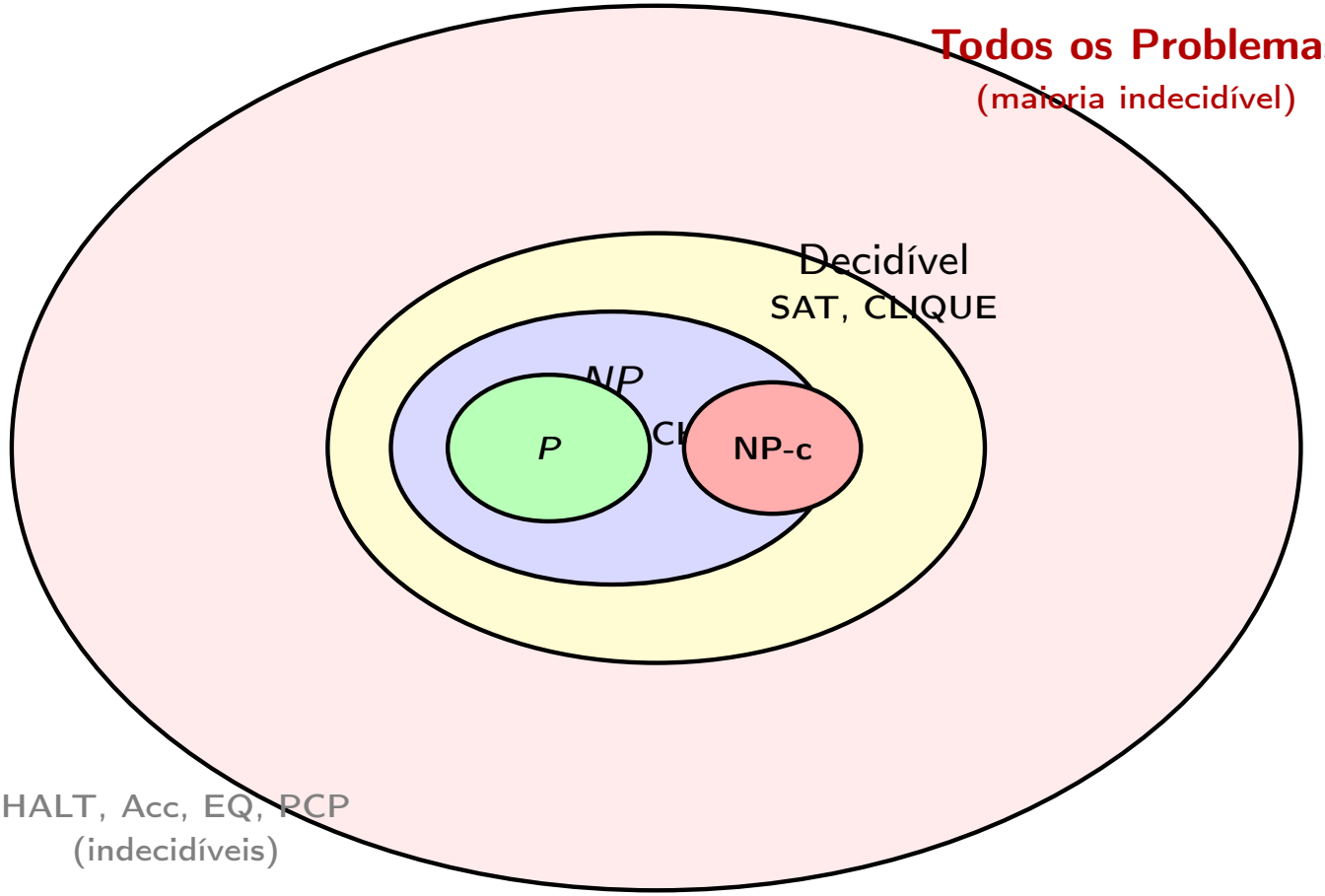
Propriedades

- $P \subseteq NP \subseteq$ Decidível.
- Existem muito mais problemas indecidíveis do que decidíveis.
- Todo problema com limite fixo de passos é decidível.

Limitação Fundamental

Não é uma limitação de hardware ou velocidade.
É uma **barreira matemática**: problemas bem definidos que nenhum computador, por mais rápido ou poderoso, pode resolver em todos os casos.

O Mapa Final das Classes de Complexidade



Resumo da Jornada

Aula 7: definimos P (resolúvel em tempo polinomial).
Aula 8: definimos NP (verificável em tempo polinomial) e NP-Completo.
Hoje: revelamos que há problemas além de qualquer algoritmo — os **indecidíveis**.

O Que Decidibilidade Significa na Prática

NP-Completo: Difícil, mas Computável

Problemas NP-completos (SAT, MOCHILA...) são **decidíveis**:
existe algoritmo (potencialmente $O(2^n)$). Em instâncias pequenas ou com heurísticas, são tratáveis.

Indecidível: Impossível em Todos os Casos

HALT, Acc, EQ: não há solução algorítmica.
Não podem ser resolvidos por nenhum algoritmo — **impossibilidade matemática**.

Implicações Reais

- **Verificação formal:** nenhuma ferramenta automática verifica corretamente *todos* os programas. Sempre há casos onde ela erra ou não termina.
- **Compiladores:** certas otimizações com garantia de equivalência semântica são impossíveis em geral (EQ indecidível).
- **Matemática:** existem afirmações verdadeiras que não podem ser provadas (Teorema da Incompletude de Gödel — conexão com indecidibilidade).

Resumo da Aula

Conceito	**Significado**
Decidível	Algoritmo que sempre para e responde certo
Indecidível	Nenhum algoritmo correto existe
HALT	Problema da Parada: indecidível (Turing, 1936)
Acc	Problema da Aceitação: indecidível
EQ	Equivalência de programas: indecidível
Diagonalização	Técnica de prova (auto-referência)
Redução $\leq_m$	Transfere indecidibilidade para novos problemas

A Hierarquia de Dificuldade

Em P $\subset$ NP $\subset$ Decidível $\subset$ Tudo

"*Polinomial*" $\rightarrow$ "*Existe Algoritmo*" $\rightarrow$ "*Impossível*".

Exercício 1 – Classificação

Exercício 1

Classifique cada problema como: **(P)**, **(NP-completo)**, **(decidível fora de NP)**, ou **(indecidível)**. Justifique em uma frase.

(i) Dado array A: está ordenado em ordem crescente?

(ii) Dado programa P e entrada x: $P(x)$ para em até 10^6 passos?

(iii) Dada fórmula 3-SAT: é satisfatível?

(iv) Dado programa P: P para para *toda* entrada possível?

Solução – Exercício 1

(i) Verificar se A está ordenado: **em P**

Scan linear $O(n)$: IS-SORTED $\in P$.

(ii) $P(x)$ para em $\leq 10^6$ passos: **em P**

Simule $P(x)$ por exatamente 10^6 passos. Para: SIM. Não para: NÃO.
Custo: $O(10^6 \cdot |P| \cdot |x|)$ — constante vezes polinomial. Está em P.
(Sem o limite de passos, seria o HALT original: indecidível.)

(iii) Fórmula 3-SAT satisfatível: **NP-completo**

Certificado = atribuição de variáveis ($O(n)$). Verificação $O(m \cdot n)$. NP-completo pelo Teorema de Cook–Levin.

(iv) P para para toda entrada: **indecidível**

Propriedade não trivial do comportamento de P, pelo Teorema de Rice. Também provável diretamente por redução de HALT (ver Exercício 4).

Exercício 2 – Analisando o Programa D

Exercício 2

Na prova de indecidibilidade de HALT, construímos $D(P)$:

- Se $H(P, P) =$ SIM: D entra em loop infinito.
- Se $H(P, P) =$ NÃO: D para e retorna SIM.

(a) O que acontece ao rodar $D(D)$, se $H(D, D) =$ SIM?

(b) O que acontece ao rodar $D(D)$, se $H(D, D) =$ NÃO?

(c) Por que isso prova que H não pode existir?

Solução – Exercício 2

(a) $H(D, D) = \text{SIM}$

H prevê que $D(D)$ para.
Pela definição de D: se $H(D, D) = \text{SIM}$, então $D(D)$ entra em **loop**.
Contradição: H disse que para, mas $D(D)$ não para.

(b) $H(D, D) = \text{NÃO}$

H prevê que $D(D)$ não para.
Pela definição de D: se $H(D, D) = \text{NÃO}$, então $D(D)$ **para** e retorna SIM.
Contradição: H disse que não para, mas $D(D)$ para.

(c) Conclusão

Em qualquer caso chegamos a uma contradição.
Logo a suposição de que H existe é **falsa**: **HALT é indecidível**. ■

A Essência

D é construído para fazer o *oposto* do que H prevê. A contradição vem da auto-referência: D pergunta ao oráculo sobre si mesmo.

Exercício 4 – Redução

Exercício 4

Mostre que o problema "P para para toda entrada" é **indecidível**, usando redução a partir de HALT.

Roteiro:

a) Dado (P_0, x_0) instância de HALT, descreva um programa P'.

b) Prove a equivalência:

$$P' \text{ para para toda entrada} \iff P_0(x_0) \text{ para.}$$

c) Conclua formalmente por que isso implica indecidibilidade.

Solução – Exercício 4

(a) Construção de P'

Dado (P_0, x_0) instância de HALT, construa P':

```
1  funcao P'(qualquerCoisa):
2    // Ignora completamente a entrada recebida
3    P0(x0)          // Executa P0 sobre x0 (entradas fixas)
4    retorne SIM
```

(b) Equivalência

(⇒) Se $P_0(x_0)$ para: P'(qualquer) termina e retorna SIM. Logo P' para para **toda** entrada.

(⇐) Se $P_0(x_0)$ não para: P'(qualquer) fica presa no loop de $P_0(x_0)$. Logo P' **não para** para nenhuma entrada.

(c) Conclusão

$P_0(x_0)$ para $\iff$ P' para para toda entrada.

Se existisse decididor D_X para "P para para toda entrada", usaríamos: dado (P_0, x_0), construa P' e consulte D_X.
Como HALT é indecidível, D_X não pode existir. ■